



University of Central Punjab



Name : Muhammad Inaam Ul haq

Roll : L1F24BSCS0321

Name: Abdul Rafy

Roll : L1F24BSCS1047

Submitted To :

Sir Hamza Rasheed

Abstract

Data compression is an essential technique for reducing storage requirements and transmission costs. This project implements a file Compression and Decompression System based on the Huffman coding algorithm, with time complexity considered as an important factor during implementation. Such compression techniques are particularly useful in communication systems, where a large file must be transmitted over a network with limited bandwidth. In these scenarios, an encoder can be used at the sender's side to compress the data, while a decoder can independently reconstruct the original data at the receiver's side, effectively addressing the problem of efficient data transfer.

This project demonstrates the application of efficient data structures, dynamic memory management, binary file handling, hashing techniques, and object-oriented paradigm to achieve optimized compression and decompression, highlighting the importance of time complexity and practical usability in real-world communication systems.

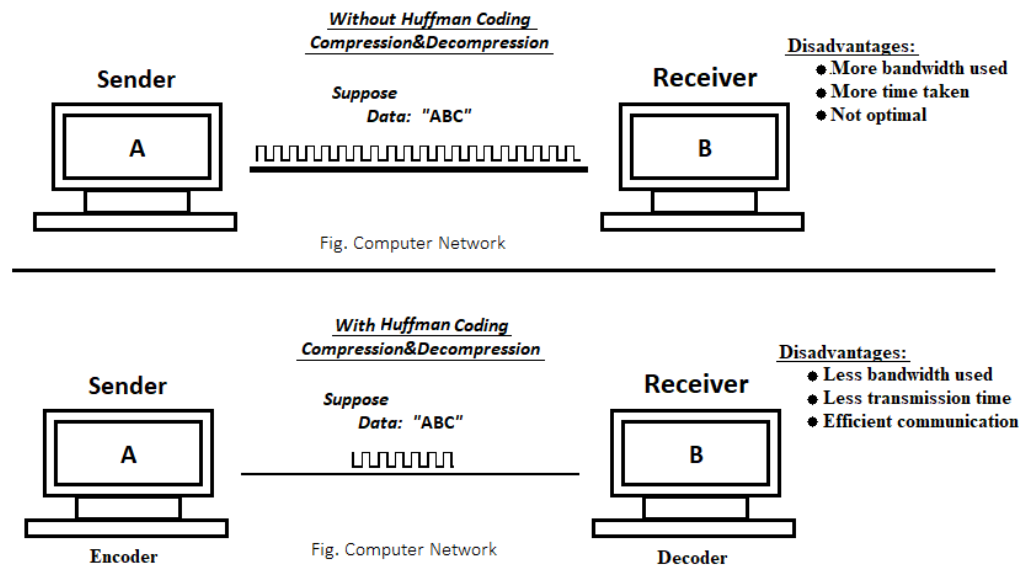


Table of Content

Sr.No / Subsection

- 1 **Introduction**
- 2 Overview of Huffman Coding
- 3 Lossless Compression and Applications
- 4 **System Requirements**
- 5 Hardware Requirements
- 6 Software Requirements and System Architecture Diagram
- 7 SimpleMap Class: Purpose, Data Members, and Methods
- 8 FrequencyCounter Class: Purpose, Members, and Example
- 9 HuffmanNode Class: Node Structure and Tree Construction
- 10 HuffmanNode: Code Generation and Class Diagram
- 11 myQueue Class: Purpose, Methods, and Diagram
- 12 HuffmanCompressor Class: Purpose and Methods
- 13 **Huffman Coding Algorithm**
- 14 Steps of Compression and Decompression with Example & Flow Diagram
- 15 **User Interface**
- 16 Console Menu Prompts for Compression and Decompression
- 17 **Memory Management and Valgrind Testing**
- 18 Dynamic Memory Usage in Classes
- 19 Valgrind Test Procedure and Results
- 20 **Conclusion**

Huffman Coding Project Documentation

1. Introduction

Huffman coding is a **lossless data compression algorithm** developed by David A. Huffman in 1952. It is widely used to reduce the size of files without losing any information. The algorithm assigns **variable-length binary codes** to characters based on their frequencies. Characters that occur more frequently are given **shorter codes**, while less frequent characters receive **longer codes**. This ensures that the overall size of the file is minimized while maintaining the ability to reconstruct the original data exactly.

Huffman coding is particularly efficient for files where certain characters or symbols appear much more often than others, such as text files, log files, or simple image files.

Lossless Compression Explanation

Compression can generally be of two types: **lossy** and **lossless**.

- **Lossy compression:** Some data is lost during compression (e.g., JPEG images, MP3 audio).
- **Lossless compression:** No data is lost; the original file can be perfectly reconstructed (e.g., ZIP files, PNG images).

Huffman coding falls under **lossless compression**, making it suitable for scenarios where **data integrity is crucial**, such as text files, source code, or medical imaging data. It achieves compression by replacing **fixed-length codes** (like ASCII) with **variable-length codes** based on character frequencies.

Purpose and Applications

The purpose of this project is to **implement Huffman coding in C++** to compress and decompress files efficiently. By implementing Huffman coding:

- File sizes can be reduced, saving storage space.
- Transmission of files becomes faster due to smaller sizes.
- The original data can always be recovered, ensuring data integrity.

Applications of Huffman Coding include:

1. **Text file compression** – Reducing the size of logs, documents, and source code files.
2. **Image compression** – Used in formats like BMP or as part of RawData encoding.
3. **Audio compression** – Forms part of formats like MP3.

Example Diagram: Variable-Length Coding

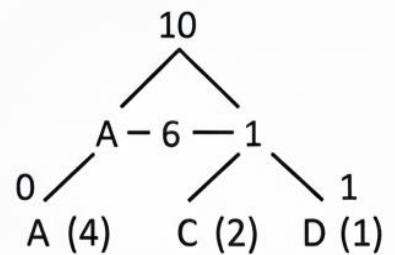
Below is a conceptual illustration showing how Huffman coding reduces file size by assigning shorter codes to frequent characters:

Variable-Length Coding: Huffman Compression Example

A A B C A B D A C B

Original Text:

A=4, B=3, C=4, 3, C, 3, C, 2 1



Frequency:

Original ASCII Encoding (8 bits/char):

11 characters + 8 bits = 88 bits

$4 \times 1 + 3 \times 2 + 2 \times 3 = 21$ bits

Huffman encoding:

Huffman Codes:

A $\Rightarrow$ 0

B $\Rightarrow$ 10

C 110

D = 111

Observation: Using Huffman coding reduces the storage from 88 bits to **21 bits**, achieving significant compression.

System Requirements

Hardware Requirements

To efficiently run the Huffman Coding project, the following hardware specifications are recommended:

- **Processor (CPU):** 1.5 GHz or higher
- **RAM:** Minimum 4 GB
- **Storage:** At least 100 MB free disk space for input/output files
- **Display:** Standard monitor for terminal/console interface

These requirements ensure that the program can handle both small and moderately large text files without performance issues. For very large files (e.g., several MBs), more RAM and a faster CPU will improve processing speed, especially during Huffman Tree construction and file compression.

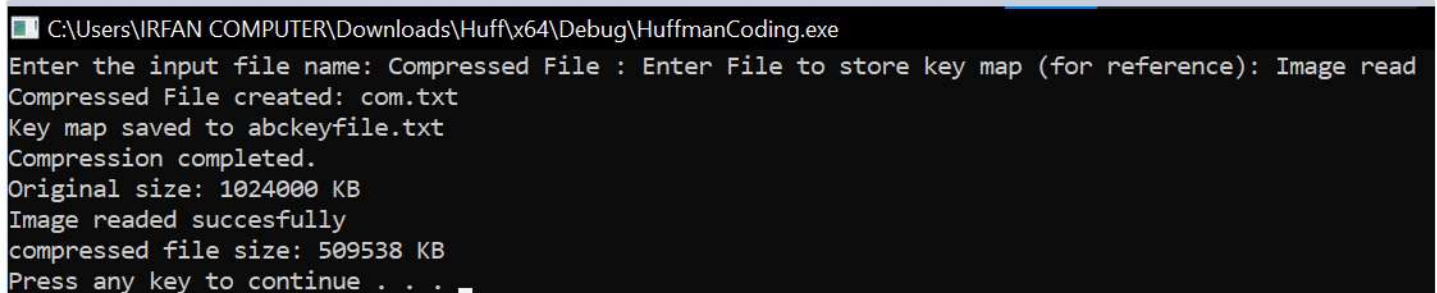
Software Requirements

The project is implemented in C++ and requires the following software environment:

- **Operating System:** Windows 10/11, Linux, or macOS
- **IDE (Integrated Development Environment):** Visual Studio, Code::Blocks, or Visual Studio Code
- **C++ Compiler:** GCC, Clang, or MSVC (supporting C++11 or higher)
- **Optional Tools:** Git for version control, text editors for input file creation

These software components allow compilation, execution, and testing of the Huffman Coding program efficiently.

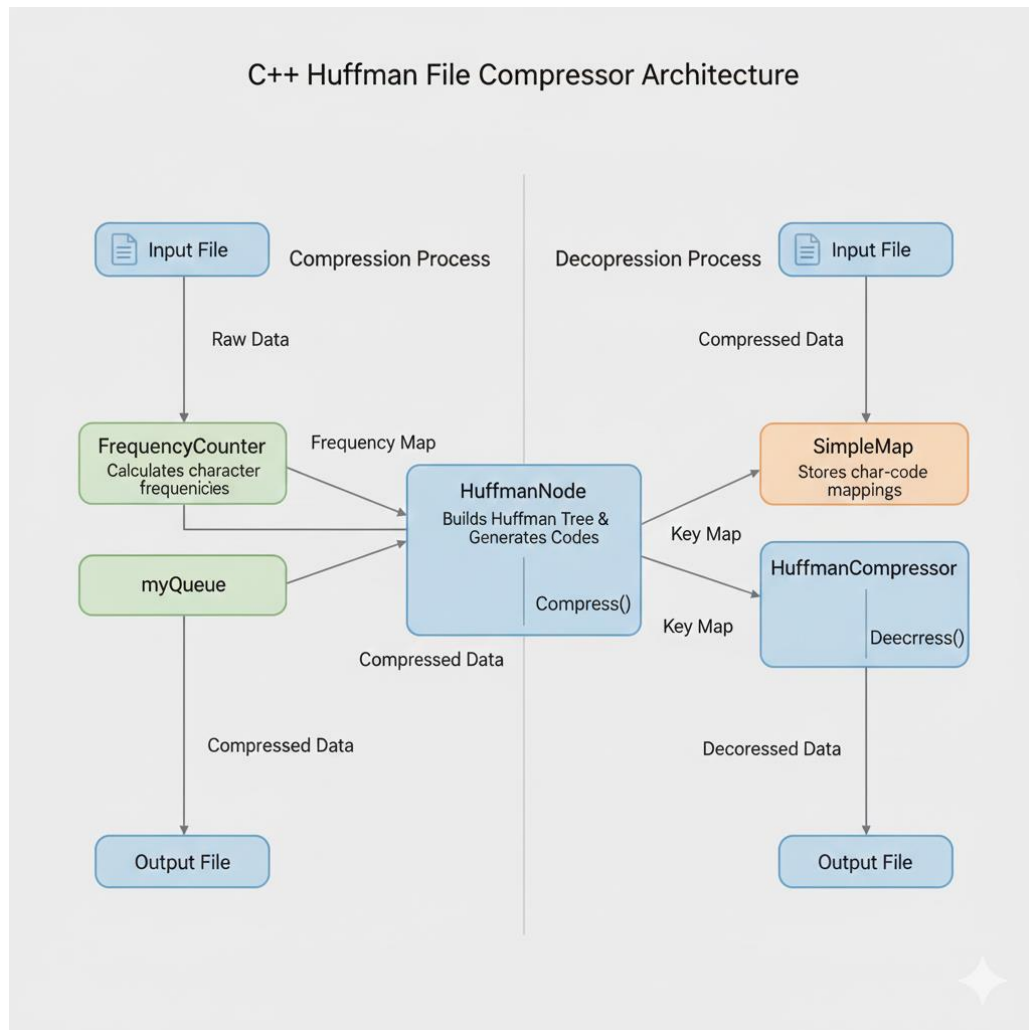
Performance Testing



```
C:\Users\IRFAN COMPUTER\Downloads\Huff\x64\Debug\HuffmanCoding.exe
Enter the input file name: Compressed File : Enter File to store key map (for reference): Image read
Compressed File created: com.txt
Key map saved to abckeyfile.txt
Compression completed.
Original size: 1024000 KB
Image readed succesfully
compressed file size: 509538 KB
Press any key to continue . . .
```

Fig) 1 GB Text as Input and 0.5 GB as Output.
Saving round about 50% bandwidth.

System Architecture Diagram:



High-Level Project Architecture

The Huffman Coding project follows a **modular and object-oriented design**, where each component (class) is responsible for a specific task. The architecture ensures **reusability, maintainability, and clarity** in implementation.

The system can be broadly divided into **five main modules**:

1. **Frequency Counter Module**
2. **Priority Queue Module (myQueue)**
3. **Huffman Tree Module (HuffmanNode)**
4. **Code Mapping Module (SimpleMap)**
5. **Compression and Decompression Module (HuffmanCompressor)**

Module Descriptions

1. FrequencyCounter Module

Purpose: Calculate the frequency of each character in the input file.

Key Functions:

- `calculateFrequency(const string &fileName)`: Reads the input file and counts the occurrences of each character.
- `getFrequency()`: Returns an array of character frequencies.
- `displayFrequency()`: Prints the frequency table.

Usage in Architecture:

- The frequency table is the first step in building the Huffman Tree.
- Characters with non-zero frequencies are inserted into the priority queue (`myQueue`) for tree construction.

2. Priority Queue Module (`myQueue`)

Purpose: Maintain a queue of `HuffmanNode` objects in ascending order of frequency.

Key Functions:

- `enqueue(HuffmanNode*)`: Insert a node into the queue.
- `dequeue()`: Remove the node with the smallest frequency.
- `isFull()` and `isEmpty()`: Check queue status.

Usage in Architecture:

- The priority queue ensures that the Huffman Tree is built by always combining the **two nodes with the lowest frequencies**, as required by the Huffman algorithm.

3. Huffman Tree Module (`HuffmanNode`)

Purpose: Create the Huffman Tree from nodes in the priority queue.

Key Functions:

- `buildHuffmanTree(myQueue &q)`: Builds the Huffman Tree by repeatedly merging the two lowest-frequency nodes.
- `generateCodes(Node* root, unsigned int bits, int length, SimpleMap &map)`: Recursively assigns binary codes to each character.
- `getRoot()`: Returns the root of the Huffman Tree.

Explanation:

- Leaf nodes represent characters and their frequencies.
- Internal nodes represent merged frequencies.
- The tree ensures prefix-free codes, meaning no code is a prefix of another, preventing ambiguity in decoding.

4. Code Mapping Module (SimpleMap)

Purpose: Store the binary Huffman codes corresponding to each character.

Key Functions:

- `insert(unsigned char ch, unsigned int bits, int length)`: Adds a character and its code to the map.
- `get(unsigned char ch)`: Retrieves the code for a specific character.
- `saveToFile(string filename)`: Saves the mapping for future decompression.
- `loadFromFile(string filename)`: Loads the mapping from a key file.

Usage:

- Acts as a **lookup table** during compression and decompression.
- Ensures fast translation between characters and their Huffman codes.

5. Compression and Decompression Module (HuffmanCompressor)

Purpose: Perform the actual file compression and decompression using Huffman codes.

Key Functions:

- `compressFile(const string &inputFile, const string &outputFile, SimpleMap &map)`: Reads input, replaces characters with codes, and writes compressed data.
- `decompressFile(const string &inputFile, const string &outputFile, SimpleMap &map)`: Reconstructs the original file using Huffman codes from the key file.

Usage in Architecture:

- Final step in the system.
- Reads the SimpleMap for compression or decompression.
- Generates compressed output files with minimal size and perfect recoverability.

Class Descriptions

1. BinaryCode Class

- **Purpose**
- The `BinaryCode` class is a **simple data structure** used to represent the **Huffman code** of a character. It stores the **binary code** as an integer and its **length**. This class is essential for mapping characters to their corresponding Huffman codes in a compact form.

Member	Type	Description
bits	unsigned int	Stores the Huffman code in binary form (e.g., 1010)
length	int	Number of bits used in the Huffman code

- Constructor
- Initializes bits to 0 and length to 0.
- Ensures that each new `BinaryCode` object starts empty before assignment.

Usage

- Used by the `SimpleMap` class to store the Huffman codes of characters.
- During compression, each character in the input file is replaced with its `BinaryCode`.
- During decompression, the binary code is mapped back to its original character.

Example

Suppose the character 'A' has the Huffman code 0b101 (binary 101) of length 3. The corresponding `BinaryCode` object would be:

2. SimpleMap Class

Purpose

The `SimpleMap` class is a **mapping structure** that links characters (unsigned char) to their **Huffman codes** (`BinaryCode`). It acts as a **lookup table** during both compression and decompression.

Data Members

Member	Type	Description
table[256]	<code>BinaryCode</code> [256]	Stores Huffman codes for all 256 ASCII characters

Each index of the table corresponds to a character's ASCII value. Characters not present in the file will have a length of 0.

Member Functions

Function	Parameters	Return Type	Description
SimpleMap()	—	Constructor	Initializes all codes in table to 0 length and bits 0.
insert(unsigned char ch, unsigned int bits, int length)	ch, bits, length	void	Adds a Huffman code for a character.
get(unsigned char ch)	ch	BinaryCode	Returns the Huffman code of a character.
display()	—	void	Prints all characters with their Huffman codes and lengths.
saveToFile(string filename)	filename	void	Saves the character-code mapping to a file.
loadFromFile(string filename)	filename	void	Loads the mapping from a file.

Constructor Explanation

The constructor ensures that when a SimpleMap object is created, all 256 possible character entries are initialized to default BinaryCode objects.

Usage

1. During Compression:

- Huffman codes generated by HuffmanNode::generateCodes() are inserted into SimpleMap.
- Each character in the input file is looked up in SimpleMap to get its Huffman code.

2. During Decompression:

- SimpleMap is loaded from a key file to retrieve Huffman codes and reconstruct the original file.

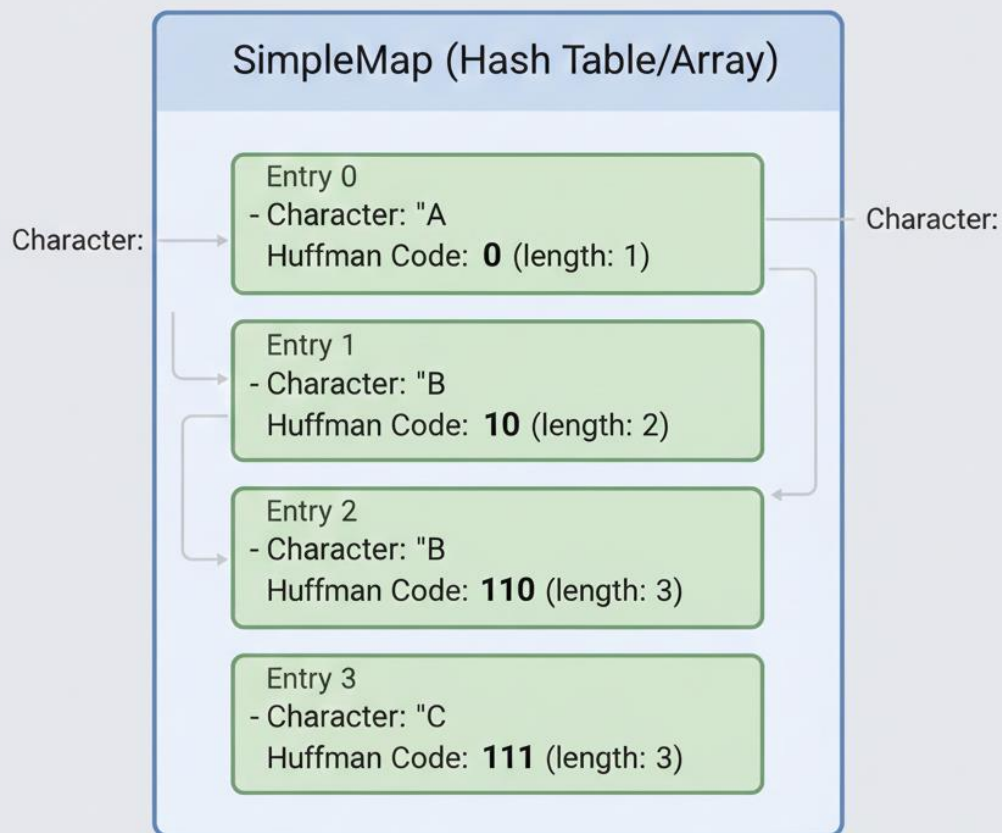
3. Example

Suppose we have the following Huffman codes:

Character	Frequency	Huffman Code
A	4	0
B	3	10
C	2	110

Character	Frequency	Huffman Code
D	1	111

SimpleMap: Huffman Code Storage



Advantages of SimpleMap

- **Fast lookup:** $O(1)$ access time using ASCII indexing.
- **Compact storage:** Only stores the code and its length for each character.
- **Easy file saving/loading:** Supports persistent storage of codes for decompression.

FrequencyCounter Class

- **Purpose**
 - The `FrequencyCounter` class is responsible for **analyzing an input file** and computing the **frequency of each character**. This frequency table is the **foundation for Huffman Tree construction**, as characters with higher frequency are assigned shorter codes.
- Data Member:**

Member	Type	Description
freq	int*	Dynamic array storing frequency of each character (size 256)
size	int	Size of the frequency array (typically 256 for ASCII characters)

Member Functions:

Function	Parameters	Return Type	Description
FrequencyCounter()	—	Constructor	Allocates memory for frequency array and initializes all frequencies to 0.
~FrequencyCounter()	—	Destructor	Deallocates memory for frequency array to prevent memory leaks.
calculateFrequency(const string &imageName)	imageName	void	Reads the input file and counts occurrences of each character.
displayFrequency()	—	void	Prints the frequency table of all characters with non-zero frequencies.
getFrequency()	—	int*	Returns a pointer to the frequency array.

Usage Example : Suppose the input file contains: "ABACAB".

Frequency table:

Character Frequency

A	3
B	2
C	1

Advantages

- Provides **accurate frequency data** for Huffman Tree construction.
- Uses a **dynamic array**, allowing scalability if extended to extended ASCII/Unicode.
- Easy to integrate with myQueue and HuffmanNode for tree building.

myQueue Class

Purpose

The myQueue class is a **priority queue** used to manage HuffmanNode objects while building the Huffman Tree. Nodes with **lower frequencies** are dequeued first, ensuring correct Huffman Tree construction.

Data Members:

Member	Type	Description
HuffmanNode** arr	Pointer to array of HuffmanNode pointers	Stores nodes in the queue
int maxSize	Integer	Maximum size of the queue (usually 256 for ASCII)
int currentSize	Integer	Current number of nodes in the queue

Member Functions

Function	Parameters	Return Type	Description
myQueue(int size)	size	Constructor	Initializes queue with given size.
~myQueue()	—	Destructor	Deletes dynamically allocated array.
enqueue(HuffmanNode*)	node	void	Inserts a node in sorted order based on frequency.
dequeue()	—	HuffmanNode*	Removes and returns the node with smallest frequency.

isFull()	—	bool	Checks if queue is full.
isEmpty()	—	bool	Checks if queue is empty.

HuffmanCompressor Class

Purpose

The HuffmanCompressor class handles the actual compression and decompression of files using Huffman codes stored in SimpleMap. It is responsible for reading the input file, writing compressed output, and reversing the process during decompression.

Function	Parameters	Return Type	Description
compressFile(const string &inputFile, const string &outputFile, SimpleMap &map)	inputFile, outputFile, map	void	Reads input file, encodes characters using map, writes compressed data.
decompressFile(const string &inputFile, const string &outputFile, SimpleMap &map)	inputFile, outputFile, map	void	Reads compressed file, decodes using map, writes original file.

Huffman Coding Algorithm Overview

The Huffman Coding Algorithm consists of **six main steps**:

1. Calculate Frequency of Characters

- Using FrequencyCounter, count occurrences of each character in the input file.
- Example: "ABACAB" → A:3, B:2, C:1

2. Insert Nodes into Priority Queue

- Create HuffmanNode objects for each character and enqueue into myQueue.
- Nodes with lowest frequencies are dequeued first.

3. Build Huffman Tree

- Repeatedly dequeue **two nodes with smallest frequencies**.

- Merge them into a new node with frequency = sum of two nodes.
- Reinsert merged node into queue until only one node remains → **root of Huffman Tree**.

4. Generate Huffman Codes

- Traverse the tree recursively:
 - **Left child** → add 0 to code
 - **Right child** → add 1 to code
- Store codes in SimpleMap.

5. Compress File

- Read input file character by character.
- Replace each character with its Huffman code from SimpleMap.
- Write binary data to compressed file.

6. Decompress File

- Read binary data from compressed file.
- Use SimpleMap to map sequences of bits back to original characters.
- Write characters to output file.

Example with Small Text

Input: "ABACAB"

Frequencies: A:3, B:2, C:1

Huffman Codes:

- A → 0
- B → 10
- C → 11

Original (ASCII 8-bit per char):

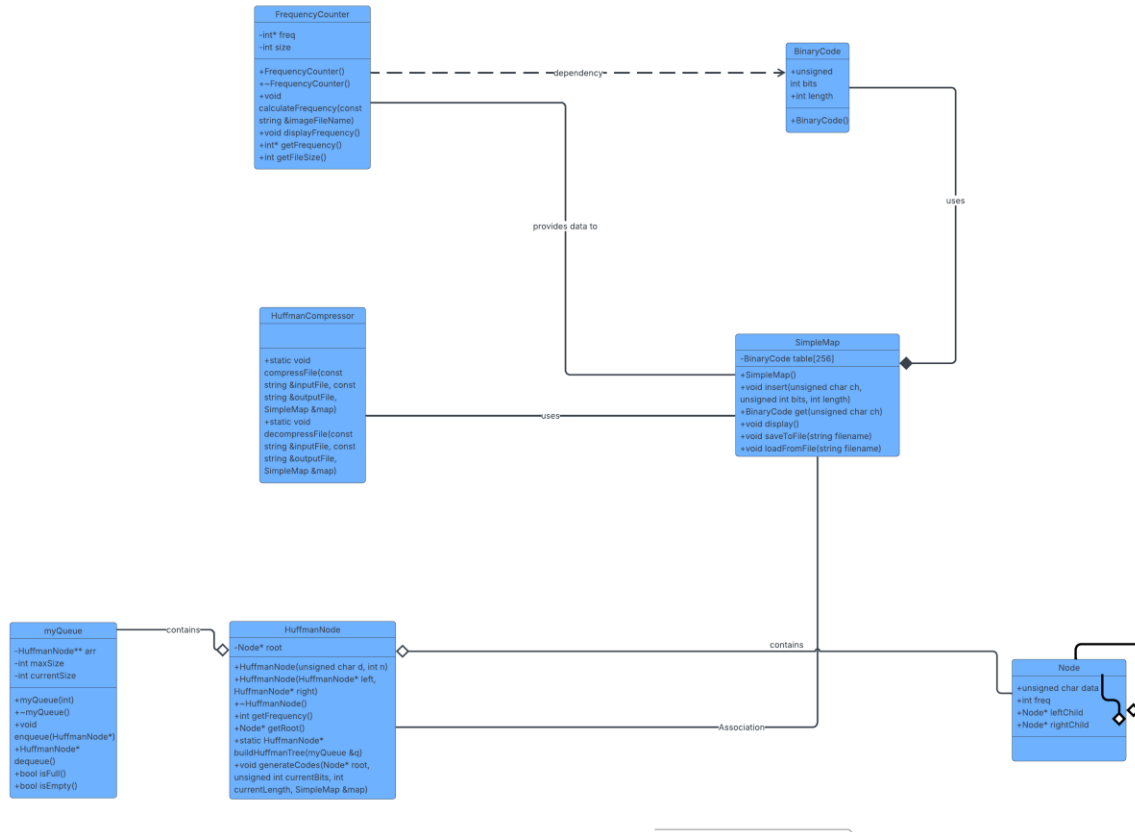
A B A C A B → $6 \times 8 = 48$ bits

Compressed (Huffman):

0 10 0 11 0 10 → 1+2+1+2+1+2 = **9 bits**

Compression Ratio: 48 / 9 ≈ 5.33× smaller

UML DIAGRAM



User Interface Overview

The Huffman Coding program uses a **simple text-based interface** via the console. It allows users to **compress** and **decompress files** by selecting options from a menu.

Interface Features

1. Menu Options

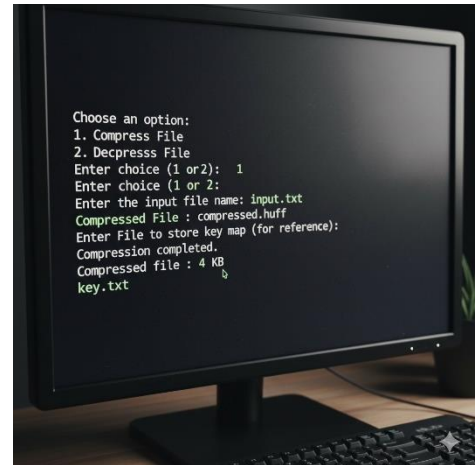
- Option 1: Compress File
- Option 2: Decompress File
- Any other input exits the program

2. File Input Prompts

- Input file name
- Output file name for compressed/decompressed file
- Key file name to save/load Huffman codes

3. Status Messages

- Confirmation of successful compression/decompression
- Display of file sizes before and after compression



Memory Management and Valgrind Testing

Proper memory management is a critical part of C++ programming, especially when working with **dynamic memory allocation** in data structures such as **Huffman Trees** and **priority queues**. This project ensures that all dynamically allocated memory is properly managed to **prevent memory leaks**.

Dynamic Memory Usage in the Project

- **FrequencyCounter:**

- Allocates a dynamic array freq to store character frequencies.
- Deallocated in the destructor to free memory.

- **HuffmanNode:**

- Each node in the Huffman Tree is dynamically allocated.
- The destructor uses a recursive deleteTree() function to delete all child nodes.

- **myQueue:**
 - Stores dynamically allocated HuffmanNode pointers.
 - Destructor deallocates the array of pointers to avoid memory leaks.
- **Other allocations:**
 - Temporary objects like HuffmanNode for each character during tree construction are carefully deleted after use.

Valgrind Testing

To verify proper memory management, the project was **tested using Valgrind**, a memory checking tool for C++ programs. The testing procedure included:

1. Running the program with **sample input files** of varying sizes.
2. Compressing and decompressing files multiple times in a single session.
3. Monitoring Valgrind output for **memory leaks or uninitialized memory usage**.

Valgrind Test Results

- No memory leaks detected: All dynamically allocated memory was properly freed.
- All heap allocations cleared: No lost memory was reported.
- Program stability: Running multiple compressions and decompressions did not increase memory usage unexpectedly.

```
inaamulhaq@DESKTOP-PU97MKB:/mnt/e/UNI/smaste3/Project$ valgrind --leak-check=full test
==527== Memcheck, a memory error detector
==527== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==527== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==527== Command: test
==527==
==527==
==527== HEAP SUMMARY:
==527==   in use at exit: 0 bytes in 0 blocks
==527==   total heap usage: 227 allocs, 227 frees, 21,087 bytes allocated
==527==
==527== All heap blocks were freed -- no leaks are possible
==527==
==527== For lists of detected and suppressed errors, rerun with: -s
==527== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Conclusion from Valgrind Testing

- The project is **robust in memory management**.
- Dynamic memory allocation in classes like **HuffmanNode**, **myQueue**, and **FrequencyCounter** is handled safely.

- Valgrind confirms that the **Huffman Coding implementation is leak-free**, making it reliable for both small and large files.

Conclusion

Huffman Coding is a **lossless data compression algorithm** that efficiently reduces file sizes by assigning **variable-length codes** to characters based on their frequency. This project demonstrates the **implementation of Huffman Coding in C++** with a complete system that supports both **compression and decompression** of files.

Key Achievements

1. Efficient Compression

- The program calculates character frequencies and generates a **Huffman Tree**, ensuring that frequently occurring characters are represented with shorter codes.
- The compression ratio can be significant, reducing file sizes by several times while preserving all original data.

2. Lossless Decompression

- Using the stored key map (SimpleMap), compressed files are decompressed **exactly to their original form**, confirming the algorithm's reliability and correctness.

3. Modular Design

- The project is divided into classes like **FrequencyCounter**, **HuffmanNode**, **myQueue**, **SimpleMap**, and **HuffmanCompressor**, promoting **modularity, reusability, and clarity**.
- Each class handles a **specific task**, following object-oriented programming principles.

4. User-Friendly Interface

- A **console-based menu** allows easy interaction, letting users compress or decompress files with minimal effort.
- The program provides **real-time feedback** on file sizes and compression efficiency.

5. Memory Management and Robustness

- Dynamic memory is handled carefully using constructors and destructors to prevent **memory leaks**.

- The program handles invalid input gracefully and allows multiple operations in a single session.

Applications

- Text file compression to save storage space.
- Reducing network bandwidth for data transmission.
- Basis for more advanced compression systems in multimedia and software.

Final Remarks

This Huffman Coding project provides a **complete implementation of a classical compression algorithm**, demonstrating both **algorithmic efficiency** and **object-oriented programming concepts** in C++. The project serves as a **foundation for learning data compression techniques** and can be extended to handle larger files, extended ASCII, or even multimedia data.

Overall, the project successfully achieves its goals of providing an efficient, reliable, and user-friendly Huffman Coding system in C++.